

UAVs Labs - Knowledge Base (2025/26 edition)

Course notes – 2025/26 labs

Contents

1 Course/lab identity	1
2 Lab map and detailed notes	1
3 How the three labs chain together	2
4 Consolidated gotchas / things to double-check before relying on this material	3

Entry point into a deep technical analysis of the three lab assignments (Parrot AR.Drone) and their DevKit material. Detailed, equation-by-equation notes for each lab live in notes/ (linked below). This file gives the lab overview, how the three labs chain together, and how they connect to the theory in [../lectures-22-23/COURSE_NOTES.pdf](#).

1 Course/lab identity

- **Course:** Unmanned Aerial Vehicles / Aeronaves Robotizadas, MSc Aerospace Engineering (MEAer), Instituto Superior Técnico, Lisboa - 2025/26 First Semester.
- **Platform:** Parrot AR.Drone 2.0 (indoor hull), controlled over its own Wi-Fi access point (192.168.1.1), via MATLAB/Simulink + the "ARDrone Simulink Development Kit" (David Escobar Sanabria & Pieter J. Mosterman, U. Minnesota / MathWorks, v1.0, Sept. 2013 - unmodified vendor devkit reused as-is across all three labs and, per the lecture notes' course logistics, across editions).
- **Deliverable format (identical for all 3 labs):** single-column PDF report, **max 15 pages**, plus the .m scripts, submitted via Fenix, using the shared cover page 25_26_UAVs_Lab_report_cover_page.docx (Instituto/course/semester header + a 3-row group-member table + instructor field - only the "Laboratory ?" number changes between submissions).
- **Grading weight** (from [../lectures-22-23/COURSE_NOTES.pdf](#)): $L = 0.2L1 + 0.4L2 + 0.4L3$, contributing 60% of the final grade $F = 0.4E + 0.6L$.
- **Question tagging convention (all labs):** **(T)** = theoretical, solved before the lab session; **(L)** = laboratory, requires simulation/experimental data collected during the session.

2 Lab map and detailed notes

Lab	Focus (per course syllabus)	Handout	Devkit entry point	Detailed notes
Lab 1	Modelling and identification of the drone	25_26_AR_Lab1.pdf (Sept. 2025)	DevKit_Lab1/.../start_here.m	notes/01-lab1.md
Lab 2	Estimation of motion variables	25_26_UAVs_Lab2.pdf (Oct. 2025)	DevKit_Lab2/.../start_here_NAV.m	notes/02-lab2.md
Lab 3	Motion control of the drone	25_26_UAVs_Lab3.pdf (Nov. 2025)	DevKit_Lab3/.../start_here_CTRL.m	notes/03-lab3.md

Supporting reference paper (required reading for Lab 2 Section 5): P. Batista, C. Silvestre, P. Oliveira, "Partial attitude and rate gyro bias estimation: observability analysis, filter design, and performance evaluation," *International Journal of Control*, 84(5), 2011 - summarized in [notes/02-lab2.md](#).

3 How the three labs chain together

The devkit itself grows incrementally across the three labs, and each lab's report is built directly on the previous one's results:

1. **Lab 1 (Modelling)** starts from the plain devkit (`start_here.m`; models: baseline, hover/position, waypoint-tracking). Its theory section (§2) has students *derive by hand* the mixer matrix, rotor-thrust allocation, hover linearization, and per-axis transfer functions that the course's Ch1/Ch2 lecture material sets up - notably, **Lab 1's Q2.2 is the concrete, worked version of the "x configuration" exercise that the Ch2 lecture slides explicitly left unsolved** (see the gotchas list in [../lectures-22-23/COURSE_NOTES.pdf](#)). Its lab sections then *experimentally identify* the height and pitch closed-loop dynamics (step response + frequency response) and compare against the six pre-identified linear models (`ssRoll`, `ssPitch`, `ssYaw`, `ssH`, etc.) that ship with the devkit and are reused, unmodified, in every later lab.
2. **Lab 2 (Estimation)** swaps in an upgraded devkit (`start_here_NAV.m`, adds a raw-navdata decoder `decode_32bit.m` at 200 Hz, and an offline **Replay** mode) that exposes raw accelerometer/gyro samples that Lab 1's devkit did not. It explicitly starts from the same accelerometer-inclinometer baseline documented in the sensors lecture notes, builds up steady-state Kalman filters and bias-augmented complementary filters axis-by-axis (pitch, then roll), and culminates in implementing the full 6-state Kalman filter from the Batista/Silvestre/Oliveira (2011) paper - the rigorous, GAS generalization of the course's own "rate-gyro-only is unobservable, add a vector measurement to fix it" worked example.
3. **Lab 3 (Control)** swaps in a third devkit variant (`start_here_CTRL.m`, adds a **Trajectory Tracking** mode with two Simulink blocks, `desired_trajectory` and `tt_controller`, deliberately left empty for students to implement). Its theory section (§2) re-derives an LQR trajectory-tracking law plus an adaptive

disturbance-rejection term (textbook instance of the Ch7 adaptive-control pattern, generalized from a scalar unknown parameter to a vector disturbance), then §3 has students implement it against straight-line and circular reference trajectories, finishing with a line-of-sight (LOS) path-following law - the concrete worked version of the "lookahead guidance" alternative that Ch8's lecture notes mention but never derive.

Across all three labs, the same in-code warning appears verbatim in the Wi-Fi control scripts: **the AR.Drone's onboard position estimate is inaccurate**, because it integrates a noisy optical-flow-derived velocity estimate (the drone has no GPS - confirmed in [../lectures-22-23/notes/03-sensors-estimation.pdf](#)). This single limitation is the throughline that motivates the whole lab sequence: Lab 1 identifies how bad the raw dynamics/estimates are, Lab 2 builds better estimators to compensate, and Lab 3's controller performance is explicitly expected to depend on how good an estimator Lab 2 produced.

4 Consolidated gotchas / things to double-check before relying on this material

- The devkit's `guide.pdf` and all `.m` scripts are unmodified since **2013** (MATLAB R2013b, Real-Time Windows Target - since renamed/deprecated to "Simulink Real-Time" in later MATLAB releases) despite being reused in the 2025/26 edition; worth confirming with the instructor whether a newer toolchain is actually expected.
- Devkit_Lab1's and Devkit_Lab3's `guide.pdf` are **byte-identical** (verified via `md5sum`) and only document the original 3 examples (baseline, hover, waypoint-tracking) - it was never updated to document Lab 3's Trajectory Tracking mode; the Lab 3 handout itself is the only documentation for `start_here_CTRL.m`, `ARDroneTTSim.slx`/`ARDroneTT.slx`, and the desired `trajectory/tt_controller` blocks.
- No lab's `.m` scripts contain numeric controller/filter gains (K , Q , R , kw , LOS Δ/V , Kalman Q/R , etc.) - by design, these are left for students to derive and tune; there is no "reference answer" in the source material for any of them. Lab 3 also reuses the symbol kw for two *different* gains (an adaptation-law gain in §2.5 vs. a vertical proportional gain in §3.1) - worth flagging explicitly if writing a report.
- The six pre-identified linear models (`ssRoll`, `ssRoll2V`, `ssPitch`, `ssPitch2U`, `ssYaw`, `ssH`) are shared, unmodified `.mat` files across all three devkits; their numeric contents were not inspected (binary) by any of the analysis forks - load them in MATLAB if a report needs the literal identified plant coefficients.
- All `.slx` Simulink models are binary (zipped XML) and were not parsed for internal block-diagram/gain content in any lab - every claim about what a model does is inferred from its filename, the referencing `.m` scripts, and the handout text, not from opening the model itself.
- `lib/getWaypoints.m`'s header comment (all 3 labs) describes a 4-field waypoint but the code actually builds a 5-column matrix - a minor, harmless vendor doc/code mismatch, not course-introduced.

See each notes/0N-labN.md file for full task-by-task detail, exact equations, and lab-specific open questions.